



Skipjack, Cipher Block Chaining Mode (Skipjack-CBC)

11th of January, 2025

INFORMATIONAL DOCUMENT 1

Disclaimer: It's not a solution

Do not, under any circumstances, “roll your own encryption” to deal with the security of critical information, unless you *really* know what you’re doing. If you’re not familiar with authenticated modes, key derivation functions, key exchange, proper methods of random number generation, and the concept of forward secrecy, read up on them before even attempting to implement a small cryptographic system. The moment you use “encryption” as a selling point, a lot of people will trust you with their lives.

Even if you’ll be the first one to eat the dog food you make, keep in mind that cryptography is not the solution to every problem. When thinking about someone who seeks to violate your privacy, ask yourself:

1. Who they’re dangerous to
2. If your opponent wields state power or connections
3. If your opponent will use them in an arbitrary fashion
4. If your opponent’s reaction to failing discovery will be worse than whatever they do when they get the message.

Considerations

This RTC Informational Document (Infodoc) concerns a symmetric encryption algorithm called “Skipjack”, in Cipher Block Chaining Mode. Skipjack was first devised by the US Government in the 1990s for voice devices. This Infodoc concerns only one mode of operation for the cipher. As of the publication of this Infodoc, no software developed by us implements this algorithm.

Given this cipher’s well-known usage within the “CLIPPER” surveillance chip developed by the NSA, it should be approached with care. An independent study by researchers from the University of Bergen and UC Berkeley attempted to derive the design principles of the cipher, and found no malicious intent in its design¹. Additionally, the actual backdoor within the chip it was used on has already been revealed². However, even if a backdoor can be ruled out, the cipher provides limited safety.

An attack was able to reverse all but one of the encryption rounds in 2005³, and it is reasonable to assume that, in the time since then, a state intelligence agency has found a way to break it. However, the process should still be fairly long, and the cipher should still provide limited security, especially considering that the civilian sector is likely to be far behind. It should be far more secure if a system using it runs it for two rounds or more, with different keys and Initialization Vectors.

Assuming that the keys used for encryption have no discernible pattern, Skipjack’s key space is well-secured, with a brute-force attack having to pick the right key among over one septillion possibilities.

While this Infodoc is mostly a reaffirmation of the US standard⁴, it provides some additional clarification, and features “pseudocode” implementations in order to make real ones easier.

1 E. Biham, A. Biryukov, and A. Shamir, “Cryptanalysis of Skipjack Reduced to 31 Rounds Using Impossible Differentials,” *Journal of Cryptology*, vol. 18, no. 4, pp. 291–311, Jul. 2005, doi: <https://doi.org/10.1007/s00145-005-0129-3>.

2 E. Roberts, “Reevaluating US Technology Export Controls,” Stanford.edu, 2024. <https://cs.stanford.edu/people/eroberts/cs181/projects/export-controls/clipper.html>

3 L. Knudsen and D. Wagner, “On the structure of Skipjack,” *Discrete Applied Mathematics*, vol. 111, no. 1, pp. 103–116, Jul. 2001, doi: [https://doi.org/10.1016/S0166-218X\(00\)00347-4](https://doi.org/10.1016/S0166-218X(00)00347-4).

4 “SKIPJACK and KEA Algorithm Specifications,” NIST Computer Security Research Center, May 29, 1998. <https://csrc.nist.gov/csrc/media/projects/cryptographic-algorithm-validation-program/documents/skipjack/skipjack.pdf>

Definitions

$x \oplus y$ - The bitwise exclusive-or of x and y

byte - A collection of 8 bits

word - A collection of 16 bits

plaintext - An unobfuscated message

ciphertext - An enciphered message

x^k - The value of x at the k th step of encipherment/decipherment

Introduction

Skipjack is a symmetric encryption algorithm, where a single cryptographic key encrypts and decrypts the same message. It operates on 64-bit data blocks (quad-words) and utilizes an 80-bit (10-byte) cryptographic key.

The core principle of its operation is the *G-permutation*, a four-round Feistel cipher, which can be understood as in a series of operations that result in a value with a relation of equality to their input. If one only works from the output, finding the original rounds is not immediately practical. Skipjack itself operates in 32 passes, featuring a series of alternating “rules”.

Additionally, to address the problem of repeated patterns in real-world sets of encrypted data, this Infodoc recommends the usage of Cipher Block Chaining as a Mode of Operation, thereby making each block in the encipherment dependent on the previous block and on an Initialization Vector.

Substitution Box

The Skipjack S-box, called the *F-Table*, is a table of 256 bytes, allowing for a straightforward lookup. It is presented as follows:

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	a3	d7	09	83	f8	48	f6	f4	b3	21	15	78	99	b1	af	f9
01	e7	2d	4d	8a	ce	4c	ca	2e	52	95	d9	1e	4e	38	44	28
02	0a	df	02	a0	17	f1	60	68	12	b7	7a	c3	e9	fa	3d	53
03	96	84	6b	ba	f2	63	9a	19	7c	ae	e5	f5	f7	16	6a	a2
04	39	b6	7b	0f	c1	93	81	1b	ee	b4	1a	ea	d0	91	2f	b8
05	55	b9	da	85	3f	41	bf	e0	5a	58	80	5f	66	0b	d8	90
06	35	d5	c0	a7	33	06	65	69	45	00	94	56	6d	98	9b	76
07	97	fc	b2	c2	b0	fe	db	20	e1	eb	d6	e4	dd	47	4a	1d
08	42	ed	9e	6e	49	3c	cd	43	27	d2	07	d4	de	c7	67	18
09	89	cb	30	1f	8d	c6	8f	aa	c8	74	dc	c9	5d	5c	31	a4
0A	70	88	61	2c	9f	0d	2b	87	50	82	54	64	26	7d	03	40
0B	34	4b	1c	73	d1	c4	fd	3b	cc	fb	7f	ab	e6	3e	5b	a5
0C	ad	04	23	9c	14	51	22	f0	29	79	71	7e	ff	8c	0e	e2
0D	0c	ef	bc	72	75	6f	37	a1	ec	d3	8e	62	8b	86	10	e8
0E	08	77	11	be	92	4f	24	c5	32	36	9d	cf	f3	a6	bb	ac
0F	5e	6c	a9	13	57	25	b5	e3	bd	a8	3a	01	05	59	2a	46

Feistel Cipher

The Feistel cipher, called the *G-permutation* G^k , is dependent on the cryptographic key. It operates on a 16-bit value, which is separated into two bytes g_1 and g_2 , with four rounds going up to g_6 . The concatenation of g_5 with g_6 is the result of the cipher.

Beginning at g_3 up to g_6 , the equation is as follows:

$$g_i = F(g_{i-1} \oplus key_{(4k+i-3) \bmod 10}) \oplus g_{i-2}$$

Where i is the byte number, $F(x)$ is the value represented in location x of the F-table, and key_y is the y th byte in the key (addressed from 0 - 9).

Following this, we can separate the cipher into four steps:

$$g_3 = F(g_2 \oplus key_{4k \bmod 10}) \oplus g_1$$

$$g_4 = F(g_3 \oplus key_{(4k+1) \bmod 10}) \oplus g_2$$

$$g_5 = F(g_4 \oplus key_{(4k+2) \bmod 10}) \oplus g_3$$

$$g_6 = F(g_5 \oplus key_{(4k+3) \bmod 10}) \oplus g_4$$

The inverse operation $[G^k]^{-1}$, used for decipherment, is as follows:

$$g_{i-2} = F(g_{i-1} \oplus key_{(4k+i-3) \bmod 10}) \oplus g_i$$

Which means the four inverse steps are as follows, beginning at g_4 down to g_1 :

$$g_4 = F(g_5 \oplus key_{(4k+3) \bmod 10}) \oplus g_1$$

$$g_3 = F(g_4 \oplus key_{(4k+2) \bmod 10}) \oplus g_2$$

$$g_2 = F(g_3 \oplus key_{(4k+1) \bmod 10}) \oplus g_3$$

$$g_1 = F(g_2 \oplus key_{4k \bmod 10}) \oplus g_4$$

Note that the key lookup is modulo 10 due to the size of the key (10 bytes).

In Pseudocode

```
function g_perm(word w, byte k, byte key[]) {
    g1, g2 = word_split(w)

    g3 = fTable[g2 ^ key[(4 * k) % 10]] ^ g1
    g4 = fTable[g3 ^ key[(4 * k + 1) % 10]] ^ g2
    g5 = fTable[g4 ^ key[(4 * k + 2) % 10]] ^ g3
    g6 = fTable[g5 ^ key[(4 * k + 3) % 10]] ^ g4

    return bytes_to_word(g5, g6)
}

function g_perm_rev(word w, byte k, byte key[]) {
    g5, g6 = word_split(w)

    g4 = fTable[g5 ^ key[(4 * k + 3) % 10]] ^ g6
    g3 = fTable[g4 ^ key[(4 * k + 2) % 10]] ^ g5
    g2 = fTable[g3 ^ key[(4 * k + 1) % 10]] ^ g4
    g1 = fTable[g2 ^ key[(4 * k) % 10]] ^ g3

    return bytes_to_word(g1, g2)
}
```


Encryption and Decryption

Skipjack makes use of four rules: A , B , and their respective inversions A^{-1} and B^{-1} . These rules are ran over the input for 32 rounds in an alternating pattern.

In Mathematical Notation

Rule A

$$w_1^{k+1} = G^k(w_1^k) \oplus w_4^k \oplus k$$

$$w_2^{k+1} = G^k(w_1^k)$$

$$w_3^{k+1} = w_2^k$$

$$w_4^{k+1} = w_3^k$$

Rule A^{-1}

$$w_1^{k-1} = [G^k]^{-1}(w_2^k)$$

$$w_2^{k-1} = w_3^k$$

$$w_3^{k-1} = w_4^k$$

$$w_4^{k-1} = w_1^k \oplus w_2^k \oplus k$$

Rule B

$$w_1^{k+1} = w_4^k$$

$$w_2^{k+1} = G^k(w_1^k)$$

$$w_3^{k+1} = w_1^k \oplus w_2^k \oplus k$$

$$w_4^{k+1} = w_3^k$$

Rule B^{-1}

$$w_1^{k-1} = [G^k]^{-1}(w_2^k)$$

$$w_2^{k-1} = [G^k]^{-1}(w_3^k) \oplus w_3^k \oplus k$$

$$w_3^{k-1} = w_4^k$$

$$w_4^{k-1} = w_1^k$$

Where k is the current step in the counter. Where w is a word, with w_n encompassing the n th 16-bit slice of the 64-bit quad-word, addressed from 1 to 4. And where w_n^{k+1} signals the n th slice of the word in the next step of the encipherment operation, with a similar notation for decipherment.

In encipherment, the input is the plaintext. The counter starts at 1, and 1 is added to it after the completion of each round. Rule A and B are run for 8 rounds each (first A , then B) for a total of 32 steps. The output is the result of the last step.

In decipherment, the input is the ciphertext. the counter starts at 32, and 1 is subtracted from it after the completion of each round. Rule B^{-1} and A^{-1} are run for 8 rounds each (first B^{-1} , then A^{-1}) for a total of 32 steps. The output is the result of the last step.

Pseudocode Encryption

```
function ruleA(quadword qw, byte counter, byte key[]) {
    w1, w2, w3, w4 = quadword_split(qw)

    w1n = g_perm(w1, counter, key) ^ w4 ^ counter
    w2n = g_perm(w1, counter, key)
    w3n = w2
    w4n = w3

    return words_to_quadword(w1n, w2n, w3n, w4n)
```

```

}

function ruleB(quadword qw, byte counter, byte key[]) {
    w1, w2, w3, w4 = quadword_split(qw)

    w1n = g_perm_rev(w2, counter, key)
    w2n = g_perm_rev(w2, counter, key) ^ w3 ^ counter
    w3n = w4
    w4n = w1

    return words_to_quadword(w1n, w2n, w3n, w4n)
}

function encrypt(quadword qw, byte key[]) {
    for counter = 1 to 32 {
        if((counter >= 1 and counter <= 8) or (counter >= 17 and counter <=
24)) {
            ruleA(qw)
        }

        if((counter >= 9 and counter <= 16) or (counter >= 25 and counter
<= 32)) {
            ruleB(qw)
        }
    }
}

```

Pseudocode Decryption

```

function ruleArev(quadword qw, byte counter, byte key[]) {
    w1, w2, w3, w4 = quadword_split(qw)

    w1n = g_perm_rev(w2, counter, key)
    w2n = w3
    w3n = w4
    w4n = w1 ^ w2 ^ counter

    return words_to_quadword(w1n, w2n, w3n, w4n)
}

function ruleBrev(quadword qw, byte counter, byte key[]) {
    w1, w2, w3, w4 = quadword_split(qw)

    w1n = g_perm_rev(w2, counter, key)
    w2n = g_perm_rev(w2, counter, key) ^ w3 ^ i
    w3n = w4
    w4n = w1

    return words_to_quadword(w1n, w2n, w3n, w4n)
}

function decrypt(quadword qw, byte key[]) {
    for counter = 32 to 1 {
        if((counter <= 32 and counter >= 25) or (counter <= 16 and counter
>= 9)) {
            ruleBrev(qw)
        }
    }
}

```

```
        }  
        if((counter <= 24 and counter >= 17) or (counter <= 8 and counter  
>= 1)) {  
            ruleArev(qw)  
        }  
    }  
}
```

Mode of Operation

The Mode of Operation concerns the transformation of an arbitrary-length plaintext into ciphertext and back again. For the sake of simplicity, this Infodoc recommends Cipher Block Chaining (CBC).

Encryption

Generate a 64-bit value (quad-word) from any sufficiently secure criteria, such as a random number generator. This value is the Initialization Vector (IV), and does not need to be secret. The IV helps obfuscate patterns within the encrypted data, so note that re-utilizing the same IV for multiple messages will result in an increased possibility of breaking the overall set if one of the original plaintexts is known.

Take an arbitrary-length plaintext and divide it into as many quad-words as necessary to encompass the whole message. The operation for each block is:

$$qw^i = SE(pt \oplus qw^{i-1})$$

Where qw^i is the i th quad-word in the chain. Where the operation begins at quad-word qw^1 , qw^0 is the Initialization Vector, SE is the Skipjack encipherment function, and pt is the plaintext block.

Decryption

In a similar fashion to encryption, separate the ciphertext into a sufficient series of quad-words. The operation for each block is:

$$qw^i = SD(qw^i) \oplus qw^{i-1}$$

Where the operation begins at quad-word qw^1 , qw^0 is the Initialization Vector, and SD is the Skipjack decipherment function.

Pseudocode Encryption

```
function encrypt_message(string message, byte key[], quadword iv) {
    string enciphered = ""
    quadword message_blocks[] = pack_quadwords(message)

    quadword lastqw = iv
    for blocks in message_blocks {
        qw = encrypt(block ^ lastqw, key)
        lastqw = qw

        string_append_quadword(enciphered, qw)
    }
}
```

```

    }

    return enciphered
}

```

Pseudocode Decryption

```

function decrypt_message(string message, byte key[], quadword iv) {
    string deciphered = ""
    quadword message_blocks[] = pack_quadwords(message)

    quadword lastqw = iv
    for blocks in message_blocks {
        qw = decrypt(block, key) ^ lastqw
        lastqw = qw

        string_append_quadword(deciphered, qw)
    }

    return deciphered
}

```